

Traceable Executable Reasoning over Query-Time Knowledge Graphs for Table–Text Question Answering

Author Names to Be Added

Affiliations and corresponding author email to be added

Abstract. Answering questions over tables and text requires preserving structural context, connecting evidence across sources, and applying the correct operations to retrieved values. A plausible reasoning path does not establish that its computation is correct or supported by the available evidence. We present Online-KG, a retrieval-augmented question answering framework that constructs a knowledge graph at query time and evaluates candidate reasoning paths through program execution. A deterministic entity anchor resolves a table-derived bridge entity from question constraints and uses it to supplement first-stage passage retrieval. The framework maintains an executable entity–relation graph and a parallel typed structural trace for tables and passages. Candidate paths are translated into Python programs or typed numerical programs and executed through a restricted graph interface that records evidence access. An evidence-aware selector combines execution outcomes, source support, and agreement among candidate answers. Each prediction exports retrieval decisions, the selected program, evidence, and path-level diagnostics for audit. We design a controlled evaluation on HybridQA, FinQA, and HiTab to examine retrieval, table structure, executable reasoning, and candidate selection alongside their computational cost. Execution success, evidence access, and the exported score-derived confidence remain diagnostic signals rather than guarantees of answer correctness.

Keywords: Knowledge graphs · Retrieval-augmented generation · Executable reasoning · Table question answering

1 Introduction

Question answering over structured and unstructured information requires a system to connect facts expressed in different forms. A table locates a value through its row, column, and potentially several levels of headers, whereas a passage expresses relationships through natural language. These sources can play complementary roles: a table may report an earlier financial value while a nearby sentence describes a subsequent change. HybridQA studies questions requiring evidence from tables and linked passages [1]; FinQA focuses on numerical reasoning over financial reports [2]; and HiTab examines reasoning over hierarchical

tables [3]. Together, these tasks motivate systems that connect evidence while retaining the structure needed to interpret it.

Consider a question asking for the percentage increase from a dividend listed in a table to a newly declared dividend mentioned in text. Answering it requires identifying the appropriate year and dividend column, linking the two values to the same company, and computing the change relative to the earlier value. Selecting a neighboring price column or reversing the denominator produces a wrong answer even when the arithmetic executes successfully. Conversely, finding the right operands is insufficient if the generated response applies the wrong operation. This example separates three requirements: preserving source context, composing evidence, and checking the resulting computation.

A retrieval-augmented system that passes evidence to a language model must still determine how the evidence supports the answer. If table cells are serialized without their full header context, their intended meaning can become ambiguous. A graph provides an explicit representation of relationships, but graph connectivity alone does not specify how to calculate a percentage change, aggregate values, or compare alternatives. Similarly, agreement among several generated answers can reflect repeated use of the same mistaken operand. These considerations motivate candidate evaluation that examines both execution behavior and the evidence consulted during execution.

We present Online-KG, a framework organized around the sequence *construct, plan, execute, and select*. For each question, the system constructs a graph from the supplied table and retrieved passages. Deterministic structural triples retain row identities and column context, including hierarchical header paths where available. Before passage enrichment, a deterministic anchor can resolve a bridge entity from a table constraint, such as a rank, and retrieve its associated passage. Semantic triples add relationships extracted from the accompanying content, and edge provenance records their sources. A parallel typed trace retains table, row, column, cell, and passage topology for audit without changing the executable graph API. Here, “online” refers to construction at query time; it does not imply continual training or access to live web information.

The system then generates multiple candidate reasoning paths and translates them into executable Python programs. Programs access the graph through a restricted interface, allowing the executor to record the graph edges they consult. Candidate selection uses execution outcomes, evidence support, source diversity, and answer agreement. The central design choice is to evaluate candidate computations after execution, with access to their outputs and recorded evidence, instead of choosing solely from textual plans. A typed numerical intermediate representation provides a complementary execution variant with explicit lookup and arithmetic operations. Neither successful execution nor a record of evidence access is treated as proof that a candidate correctly answers the question.

Our study asks whether this combination improves answer quality under matched context and model settings, which components account for any observed changes, and what computational cost they introduce. We plan controlled comparisons on HybridQA, FinQA, and HiTab, covering heterogeneous

evidence, numerical computation, and hierarchical table structure. Initial experiments use benchmark-supplied tables and associated context; their scope is reasoning within that context rather than open-domain retrieval. Comparisons include direct answering, graph paths without code execution, agreement-only selection, and alternative executable representations.

The paper develops four methodological contributions:

- A two-stage retrieval procedure that uses inspectable question constraints to identify table-derived bridge entities, followed by entity-centered passage supplementation.
- A query-time graph construction procedure combining structural table triples and semantic evidence, with a parallel typed topology trace and edge provenance available for audit.
- An executable reasoning pipeline that converts candidate graph paths into programs and records their outputs and consulted graph evidence.
- An evidence-aware candidate selection mechanism and prediction trace that expose source support, executed paths, and answer diagnostics for controlled comparison with selection based on agreement alone.

2 Related Work

To complete: review and cite primary papers on table-text QA, retrieval-augmented generation with graphs, program-aided reasoning, and execution-based candidate selection. Compare graph construction time, evidence representation, execution interface, and evaluation setting. Do not infer novelty from an implementation difference alone.

3 Problem Formulation

Given a question q , a table T , and associated passages D , the system produces an answer a and an execution trace. In the initial benchmark setting, T and the associated passage pool are provided by the dataset. Passage ranking therefore operates within supplied context; this is not an evaluation of open-domain retrieval. The graph G_q is constructed separately for each query.

4 Method

4.1 Overview and Retrieval Context

Figure 1 summarizes the complete pipeline and the bounded replanning loop. Given a question q , supplied tables $\mathcal{T}$, and an associated passage pool $\mathcal{D}$, Online-KG returns an answer together with its selected program and recorded graph evidence. The pipeline constructs one graph per question, proposes candidate plans, executes their programs, and ranks the resulting candidates. A reasoning path here is a sequence of goals and step dependencies; it need not correspond

to a single simple path in the graph. Arithmetic may combine values from disconnected graph components that share a report context.

Retrieval precedes graph construction. A first stage ranks passages by BM25 against q . In parallel, a deterministic anchor reads a limited set of question constraints and table rows. Its current rules identify ordinal rank constraints, recognize a small set of entity-bearing columns, and resolve their matching row entity. For example, a question about the “second” player can yield the bridge entity *Walter Payton* from a rank–player table row even though the name does not occur in the question. The second stage queries this bridge entity first, preferring exact normalized entity-token mentions, then supplements the selected passages with a bounded number of items from deterministic query expansions based on question cues and table headers. The implementation uses budgets k and b for these stages, respectively, and deduplicates items by source identifier. Up to two preceding passages are provided as antecedent-resolution context for supplementary items. Tables are retained from the supplied input up to the configured table limit; this step does not perform corpus-wide table search. The Python pipeline defaults to $k = 10$ and $b = 3$; experimental settings must record any runner overrides. The anchor is deliberately rule-based and only supports the implemented constraints; a missing or wrong anchor leaves the second stage dependent on generic expansions. Optional web snippets follow the same retrieval interface, but web access is not required by the benchmark setting. Each run records the anchor fields, bridge-entity queries, expansion queries, and retrieval budgets in its retrieval trace.

4.2 Structural and Semantic Graph Construction

Let $G_q = (V_q, E_q)$ be a directed multigraph. An edge is represented as

$$e = (h, r, t, \pi), \quad (1)$$

where h and t are entity or literal nodes, r is a normalized relation, and π records provenance. Provenance includes a source type and identifier and, for structural table edges, row position, column name, and header path. Parallel edges retain alternative source records for the same relationship. Alongside G_q , the implementation stores a non-executable structural trace H_q : Table $\rightarrow$ Row $\rightarrow$ Cell $\rightarrow$ Column edges for each supplied table, plus Passage nodes for selected text. This trace retains table topology and cell locations without inserting implementation identifiers into G_q ’s entity-centric query interface.

Structural table edges. Dataset adapters provide row dictionaries with qualified column names. The extractor uses the first field as the row identifier and converts each remaining nonempty cell into an edge:

$$e_{ij} = (u_i, \text{norm}(H_j), v_{ij}, \pi_{ij}), \quad (2)$$

where u_i identifies row i , H_j is the qualified header for column j , and v_{ij} is its cell value. For hierarchical input, the adapter must retain the relevant header levels

Table 1. Illustrative input with a two-level column header. The passage supplies an additional dividend value for the same report context.

Quarter ended	2002		
	High	Low	Dividend
March 31	26.50	22.92	0.450

before extraction. For example, 2002 / Dividend becomes 2002_dividend, distinguishing it from 2002_high. The extractor records header components from this qualified name. Empty row identifiers and empty cell values are skipped; normalized self-loops are rejected by the graph layer. Consequently, preservation is conditional on the adapter representation and these insertion rules, rather than a claim that every original cell is always retained.

Semantic enrichment and resolution. An LLM optionally extracts additional table relationships and extracts triples from selected passages. Structural and semantic edges are inserted into the same graph. If optional table enrichment fails, construction falls back to structural extraction for that table; a failed passage extraction is logged and that source is skipped. Entity resolution applies lexical normalization, configured aliases, structural name rules, and a conservative similarity stage. The default similarity function compares normalized strings, with threshold 0.93 for the final stage; it is not an embedding model. Resolution events are logged. Source identifiers are preserved, but identical row labels across multiple tables can still be ambiguous because source metadata is not itself a unique node identifier. Node-type metadata currently labels graph heads as entities and tails as attributes; it is descriptive trace metadata rather than a learned ontology or a type checker.

Running example: dividend change. Table 1 and a short passage illustrate the representation. All examples in this section are constructed demonstrations, not reported benchmark predictions. Suppose the question is: *What is the percentage increase from the March dividend to the newly declared dividend?* The associated passage states: “Acme declared a new dividend of 0.455 dollars per share.” The earlier value is 0.450, whereas 26.50 is a price and must not be used as a dividend operand.

Figure 2 separates graph edges from program data flow. The table row and company node need not be connected by an invented ownership edge: the candidate program can query both components within the supplied report context. An incorrect company association remains a potential planning or extraction error.

4.3 Candidate Planning and Executable Programs

The planner receives the question, a bounded graph summary, and numerical operation hints inferred from the question and graph schema. The current summary

includes up to 50 node names and 50 relations. It requests N distinct candidate plans, each with step goals and optional dependencies. These plans are jointly generated candidates, not statistically independent samples. The code synthesizer then translates each plan p_i into a program c_i using the available graph schema and query interface.

Forward lookup, `get_neighbors`, returns objects of matching outgoing edges; reverse lookup, `get_sources`, returns matching subjects. Programs may also inspect relations and filter entity lists. The synthesizer validates generated Python, including import rejection and checks on recognized API calls. The executor exposes a restricted set of built-ins and enforces a five-second default timeout. Its output is read from the variable `result`. These restrictions are implementation safeguards, not a formally verified isolation boundary.

For the illustrative graph, the following program reads both operands from edges rather than embedding the answer:

```
old = float(kg.get_neighbors(
    "March 31", "2002_dividend")[0])
new = float(kg.get_neighbors(
    "Acme", "declared_dividend")[0])
result = (new - old) / old * 100
```

Listing 1.1. Illustrative Python program for the dividend graph.

This example assumes each lookup has one matching value and the operands share a unit. Its result is

$$100 \frac{0.455 - 0.450}{0.450} = 1.111 \dots \text{ percent.} \quad (3)$$

The multiplier 100 is an operation constant. Missing values, multiple conflicting values, zero denominators, and incompatible units require explicit handling in a generated program; the illustrative indexing is not a general ambiguity-resolution procedure.

Contrasting executable candidates. Figure 3 extends the running example with three constructed plans and their programs. A uses the dividend values with the intended operation, B reverses the subtraction, and C substitutes the high-price cell for the earlier dividend. Executing the programs produces different answers, but each accesses two edges from the same two source types. When evaluated together, all three receive score 7.25: their agreement term is $1/3$, and the remaining support and length terms are identical. Candidate order breaks the tie; this is not evidence that the evaluator detects either error. The single-candidate score reported below instead uses agreement 1.

Example: relational composition. For an illustrative question asking which country a club’s player was born in, consider the triples (Lina, club, Orion FC) from a table and (Lina, born_in, Porto) and (Porto, country, Portugal) from a passage. A program first uses reverse lookup to find Lina from Orion FC, then follows `born_in` and `country`. This example requires a join and a relation chain rather

than arithmetic. The terminal answer, Portugal, occurs directly as an edge endpoint, unlike the derived percentage in the first example.

4.4 Execution Traces and Typed Numerical Variant

Execution produces $x_i = (s_i, y_i, \mathcal{E}_i)$: a success indicator, output, and set of recorded evidence references. A tracing wrapper records provenance for edges returned by forward and reverse lookups. Returned neighbor values are deduplicated, while their parallel supporting edges remain in the trace. The trace covers returned edges even if subsequent Python computation discards some of them. Thus, $\mathcal{E}_i$ records access, not full operand-level causal dependence.

The numerical variant replaces Python generation with a JSON program containing ordered steps (id, operator, arguments, unit) and a result reference. Validation checks the operator set, step identifiers, and references to earlier steps. Lookup and constant steps supply inputs to arithmetic, comparison, aggregation, and label-selection operators. For the dividend example, its conceptual sequence is

$$\begin{aligned} v_1 &= \text{lookup}(\text{March 31, 2002_dividend}), \\ v_2 &= \text{lookup}(\text{Acme, declared_dividend}), \\ v_3 &= \text{subtract}(v_2, v_1), \quad v_4 = \text{divide}(v_3, v_1), \\ v_5 &= \text{multiply}(v_4, 100). \end{aligned} \tag{4}$$

This notation illustrates dependencies rather than the literal JSON schema. The interpreter records intermediate values and operators. Unit annotations and explicit conversions aid inspection but do not constitute a general proof of dimensional correctness. Hierarchical-table questions can additionally use label-value selection: for values (North, 12) and (South, 18), an argmax operation returns South, rather than only 18. The operator vocabulary also includes count, range, threshold filters, and rank-based selection; support for an operator does not ensure correct operand selection.

4.5 Evidence-Aware Candidate Selection

The implemented selector first rejects execution failures, empty outputs, and candidates without evidence. A direct-support heuristic matches normalized output terms to evidence endpoints using equality, token membership, or substring containment. Let $\mathcal{D}_i \subseteq \mathcal{E}_i$ denote the matched references. An output with $\mathcal{D}_i \neq \emptyset$ is classified as directly supported. Numeric or Boolean outputs with nonempty evidence may instead be admitted as derived results. This latter gate does not verify that every operand was obtained from the trace.

Outputs are grouped by the evaluator’s serialization key: scalar values use JSON serialization, and sequence-like values use sorted string elements. Grouping is not general semantic or numeric-tolerance equivalence. For an admissible output group y , evidence is pooled across its admitted candidates:

$$\mathcal{E}(y) = \bigcup_{i: \text{key}(y_i) = \text{key}(y)} \mathcal{E}_i. \tag{5}$$

Let $n_s(y)$ count distinct (source type, source id) pairs and $n_t(y)$ count distinct (h, r, t) triples in this union. Their saturated support terms are

$$U(y) = \min\{n_s(y)/3, 1\}, \quad T(y) = \min\{n_t(y)/4, 1\}. \quad (6)$$

Let $I_{\text{tab}}, I_{\text{txt}}, I_{\text{web}}$ indicate which source types occur in the group. The provenance bonus is

$$B(y) = 2I_{\text{tab}} + I_{\text{txt}} + 0.25I_{\text{web}} + 1.5I_{\text{tab}}I_{\text{txt}}. \quad (7)$$

These are fixed heuristic weights in the current implementation, not learned probabilities or guarantees of source independence.

Agreement $A(y)$ is the fraction of successful, nonempty candidates with the same output key. In the implementation this denominator is formed before the evidence gate, so it can include candidates subsequently rejected for lacking evidence. The per-candidate direct-match term is

$$P_i = \begin{cases} |\mathcal{D}_i| / \max\{a_i, |\mathcal{E}_i|, 1\}, & \text{direct support,} \\ 0.5, & \text{derived numeric or Boolean output,} \end{cases} \quad (8)$$

where a_i is the recorded accessed-edge count. Although the implementation labels this quantity “evidence precision,” it is a matching heuristic rather than precision against annotated gold evidence. With L_i planned steps, the final score is

$$S_i = 4U(y_i) + B(y_i) + 1.5T(y_i) + A(y_i) + P_i - 0.5L_i. \quad (9)$$

Rejected candidates receive $-\infty$. The highest-scoring admitted candidate is selected; ties follow candidate order. Because support is pooled by output, a selected candidate can benefit from evidence accessed by another candidate with the same answer. Its own returned trace remains $\mathcal{E}_i$.

For the dividend example, assume one admitted three-step plan accesses exactly the two operand edges, one table source and one text source. Then $U = 2/3$, $T = 1/2$, $B = 4.5$, $A = 1$, and $P = 0.5$, giving $S \approx 7.92$. This calculated score illustrates the rule, not a measured model prediction. Reversing the subtraction while accessing the same edges could obtain the same score. The example makes explicit why evidence-aware selection cannot alone establish semantic correctness.

4.6 Failure Handling and Answer Production

A synthesis or execution failure rejects the affected candidate while allowing other candidates to compete. If the Python mode yields no admissible candidate, a symbolic graph-search fallback may supply additional executed candidates. This fallback is disabled in the numerical-IR variant. If selection still fails, the pipeline generates a fresh set of plans over the same graph, up to R replanning rounds. It does not retrieve again or rebuild the graph during these rounds, and candidates are not accumulated across rounds. An empty graph or exhausted budget yields no answer and an error record.

For a selected numeric scalar, answer production returns its executed value without LLM rewriting. Boolean values are rendered as yes/no. Other outputs can pass through a verbalizer, with a normalized scalar check and fallback to the preferred observed entity alias. The returned artifact includes the anchor, retrieval trace, executable and structural graph traces, chosen program, evidence, candidate diagnostics, and path metrics (completion, planned length, evidence count, observed source types, cross-modal flag, and direct-match ratio). The artifact also exports a score-derived value in $[0, 1]$ under the field `confidence`; it is a rescaled selector score and is not a calibrated probability. Experiments must report N , R , retrieval budgets, execution mode, and fallback policy, since each changes either the candidate pool or computational cost.

5 Experimental Protocol

5.1 Datasets and Metrics

Use HybridQA for table-text QA, FinQA for table-text numerical reasoning, and HiTab for cross-domain hierarchical-table reasoning. Add verified dataset citations, versions, split sizes, and sample identifiers. Report HybridQA exact match and token F1. For FinQA, prioritize the official execution metric and separately label any relaxed ratio/percentage metric. Do not report Python program accuracy as equivalence to the official FinQA program metric. Keep normalized official tables and raw-table parsing experiments separate.

5.2 Baselines and Ablations

Compare direct answering from the same context, flattened-table retrieval, graph paths without execution, agreement-only selection, and the full system. Evaluate typed numerical programs separately. Planned construction ablations include structural-only and semantic-only triples and removal of header hierarchy. Add a retrieval ablation that sets the second-stage budget to zero, and compare generic schema expansion with bridge-entity supplementation where anchorable questions are available. Report anchor coverage, bridge-entity retrieval recall, and answer metrics separately; retrieving a bridge entity does not ensure a correct final answer. Verify that each ablation is implemented before running it.

5.3 Controls and Cost

Fix example identifiers, model version, decoding settings, context limits, and retrieval settings across comparable methods. Record candidate count, replanning budget, fallback policy, API attempts, logical model calls, and wall time. Use measured provider token usage if available; missing usage is not zero. Report paired uncertainty estimates and failure rates on a fixed evaluation set. Archive retrieval traces, typed structural traces, selected-path metrics, and the repository revision. Treat the exported confidence as an uncalibrated diagnostic; do not evaluate it as calibrated confidence without a dedicated calibration study.

6 Results and Error Analysis

Pending benchmark evaluation. The currently inspected development artifact contains one FinQA example with execution accuracy 0, 14 logical model calls, and wall time 173.55 seconds. This is a debugging observation, not an estimate of benchmark performance. It does not contain the official execution metric. Model identity must be checked in run metadata rather than inferred from the filename. Record the repository revision for future experiments.

Analyze errors in table parsing, evidence retrieval, entity resolution, operator selection, arithmetic, units, and answer rendering. Distinguish code execution success from answer correctness. Add a case study showing the exact question, source cells, graph edges, generated program, and executed result.

7 Limitations

The current experiments assume dataset-supplied tables and, for FinQA, the associated report context; they therefore do not measure open-domain table or document retrieval. HiTab is evaluated as hierarchical table-only QA, so it tests structural and numerical reasoning but cannot demonstrate table-text provenance diversity. The deterministic table graph preserves row and column header paths, but graph size grows with the number of cells, while optional LLM enrichment can introduce redundant parallel edges or noisy relations.

The numerical IR currently covers a practical but incomplete operator set. Questions involving ranking, ranges, dates, complex percentages, implicit unit conversion, or domain-specific financial operators may require additional operators or repair. A successfully executed program is not necessarily a semantically correct one: it may select the wrong grounded cells, reverse an ordered subtraction, or apply a valid operator to an incorrect set of values. Consequently, execution success is reported separately from denotation, operator, step, and grounding accuracy. Step-level evaluation also depends on the availability and alignment of gold programs and is not directly comparable across datasets with different annotation conventions.

Planning and graph enrichment still require multiple model calls, increasing latency and exposing results to provider failures and model variance. Source counts are only an approximation of independence because different passages or edges may repeat the same underlying fact. Full graph traces improve auditability but can become visually dense and should not be treated as proof that every extracted edge is correct. Finally, restricted Python execution is an engineering sandbox rather than a formally verified security boundary.

8 Conclusion

This draft defines a study of executable reasoning over query-time table-text graphs. Final conclusions will depend on controlled benchmark and ablation results.

References

1. Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., Wang, W.Y.: HybridQA: A dataset of multi-hop question answering over tabular and textual data. In: Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 1026–1036. Association for Computational Linguistics (2020). <https://doi.org/10.18653/v1/2020.findings-emnlp.91>
2. Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., Moussa, R., Beane, M., Huang, T.H., Routledge, B., Wang, W.Y.: FinQA: A dataset of numerical reasoning over financial data. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 3697–3711. Association for Computational Linguistics (2021). <https://doi.org/10.18653/v1/2021.emnlp-main.300>
3. Cheng, Z., Dong, H., Wang, Z., Jia, R., Guo, J., Gao, Y., Han, S., Lou, J.G., Zhang, D.: HiTab: A hierarchical table dataset for question answering and natural language generation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics (2022). <https://doi.org/10.18653/v1/2022.acl-long.78>

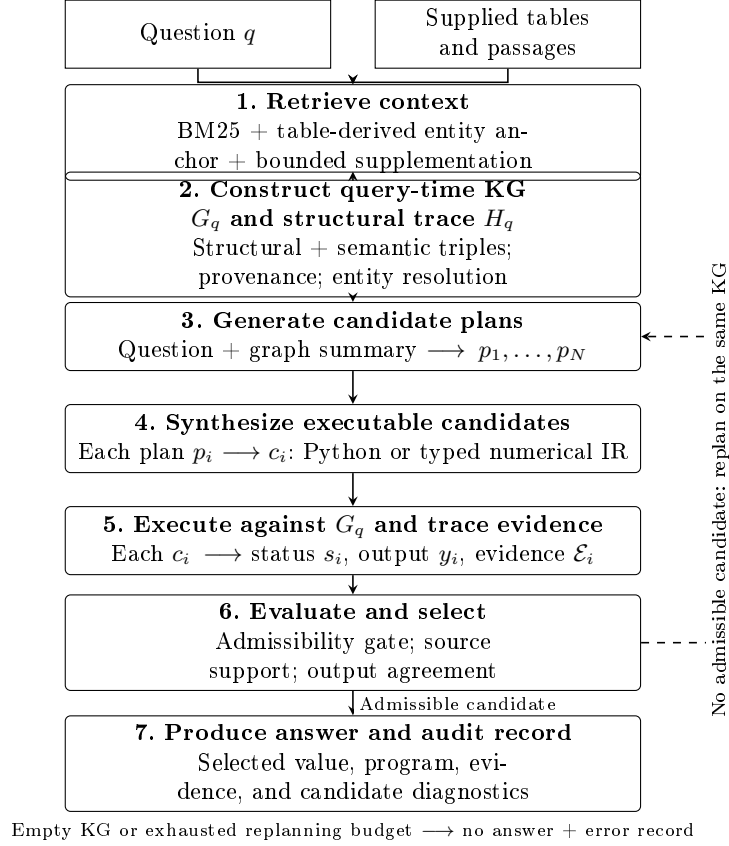


Fig. 1. Overview of Online-KG. Retrieval and graph construction run once per question. Candidate plans are converted to Python programs or, in the numerical variant, typed IR programs. Execution returns outputs and accessed-edge evidence for candidate selection. The dashed loop generates fresh plans over the same graph when no candidate is admissible, subject to a replanning budget. Python mode additionally permits a symbolic-search fallback before replanning (omitted from the diagram).

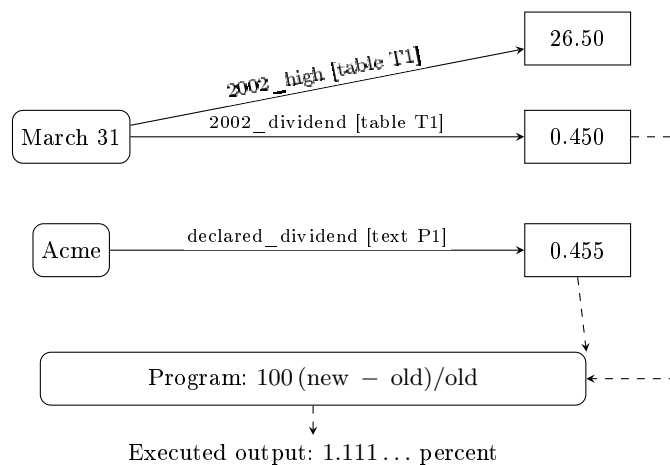
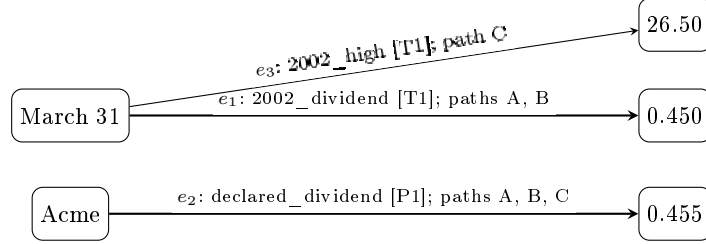


Fig. 2. Illustrative query-time KG and arithmetic data flow. Solid arrows are stored KG edges, labeled with relation and source identifier. Dashed arrows are program dependencies, not KG relations. The price edge is a distractor; only dividend edges supply operands to the displayed computation.

Question: What is the percentage increase from the March dividend to the newly declared dividend?



Solid arrows: KG edges. T1: table source; P1: text source. All values belong to the same illustrative report context.

**Path A: correct
operands**

Read e_1 , read e_2 ;
compute $(new - old)/old$.

```

a = kg.get_neighbors(
    "March 31",
    "2002_dividend")[0]
b = kg.get_neighbors(
    "Acme",
    "declared_dividend")[0]
a, b = float(a), float(b)
result = (b-a)/a*100

```

Output: +1.111...%
Evidence: $\{e_1, e_2\}$
Execution: success
Score: 7.25

**Path B: wrong
operation**

Read e_1 , read e_2 ;
reverse subtraction.

```

a = kg.get_neighbors(
    "March 31",
    "2002_dividend")[0]
b = kg.get_neighbors(
    "Acme",
    "declared_dividend")[0]
a, b = float(a), float(b)
result = (a-b)/a*100

```

Output: -1.111...%
Evidence: $\{e_1, e_2\}$
Execution: success
Score: 7.25

Path C: wrong operand

Read e_3 , read e_2 ;
use price as old dividend.

```

a = kg.get_neighbors(
    "March 31",
    "2002_high")[0]
b = kg.get_neighbors(
    "Acme",
    "declared_dividend")[0]
a, b = float(a), float(b)
result = (b-a)/a*100

```

Output: -98.283...%
Evidence: $\{e_3, e_2\}$
Execution: success
Score: 7.25

Execution success + accessed evidence do not guarantee correctness.

Fig. 3. Constructed dividend KG with three reasoning plans and their executable programs. Each plan combines two edge lookups and an arithmetic operation; it is not a simple connected graph path. Outputs and scores were reproduced with the repository executor and evaluator. A is correct for the stated question, but all candidates tie under the current heuristic. The figure illustrates a failure to distinguish operation and operand errors, not a successful ranking or a benchmark result.